

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ  
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»**

**Лабораторная работа №2  
по курсу «Программирование графических процессоров»**

**Классификация и кластеризация изображений на GPU.**

Выполнил: О.С. Концебалов

Группа: М8О-409Б-22

Преподаватели: А.Ю. Морозов,  
Е.Е. Заяц

Москва, 2025

## Условие

**Цель работы:** научиться использовать GPU для классификации и кластеризации изображений. Использование константной памяти и одномерной сетки потоков.

Формат изображения соответствует формату, описанному в лабораторной работе 2. Во всех вариантах, в результирующем изображении, на месте альфа-канала должен быть записан номер класса (кластера), к которому был отнесен соответствующий пиксель. Если пиксель можно отнести к нескольким классам, то выбирается класс с наименьшим номером.

**В вариантах 1-4, формат входных данных одинаковый.** На первой строке задается путь к исходному изображению, на второй, путь к конечному изображению. На следующей строке, число  $nc$  – количество классов. Далее идут  $nc$  строчек, описывающих каждый класс. В начале  $j$ -ой строки задается число  $np_j$  – количество пикселей в выборке, за ним следуют  $np_j$  пар числе – координаты пикселей выборки.  $nc \leq 32$ ,  $np_j \leq 2^{19}$ ,  $w \cdot h \leq 4 \cdot 10^8$ .

Оценка вектора средних и ковариационной матрицы:

$$avg_j = \frac{1}{np_j} \sum_{i=1}^{np_j} ps_i^j$$

$$cov_j = \frac{1}{np_j - 1} \sum_{i=1}^{np_j} (ps_i^j - avg_j) * (ps_i^j - avg_j)^T$$

где  $ps_i^j = (r_i^j \ g_i^j \ b_i^j)^T$  –  $i$ -ый пиксель из  $j$ -ой выборки.

## Вариант 4. Метод спектрального угла.

Для некоторого пикселя  $p$ , номер класса  $jc$  определяется следующим образом:

$$jc = \underset{j}{\operatorname{argmax}} \left[ p^T * \frac{avg_j}{|avg_j|} \right]$$

## Пример:

| Входной файл                                                       | hex: in.data                                                                                                | hex: out.data                                                                                               |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| in.data<br>out.data<br>2<br>4 1 2 1 0 2 2 2 1<br>4 0 0 0 1 1 1 2 0 | 03000000 03000000<br>A2DF4C00 F7C9FE00 9ED84500<br>B4E85300 99D14D00 92DD5600<br>A9E04C00 F7D1FA00 D4D0E900 | 03000000 03000000<br>A2DF4C01 F7C9FE00 9ED84501<br>B4E85301 99D14D01 92DD5601<br>A9E04C01 F7D1FA00 D4D0E900 |

| Входной файл                                                                                                                                          | hex: out.data                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in.data<br>out.data<br>5<br>4 5 0 0 2 6 1 1 1<br>6 2 0 7 1 1 0 1 2 6 0 4 0<br>4 3 0 3 1 0 1 0 0<br>4 0 3 6 2 5 2 7 2<br>9 6 4 5 1 7 0 2 1 2 3 4 1 1 5 | 08000000 08000000<br>D2E27502 CFF65201 D3ED5701 D6E76902<br>C8F35B01 8E168200 CFF45001 AE977604<br>D3DC7102 7D1E7B00 AB9A8004 D9E58602<br>AB967E04 AE9D8004 87058200 D0F95B01<br>74148000 D0F55901 86136C00 85077400<br>D6E27702 D3609F03 D1609F03 CC5EA103<br>CC739D03 7C127F00 AA988804 AFA07D04 |

|         |                                                                                                                                                                                                                                                                                                                                                             |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3 3 2 6 | D0E37702 7D117A00 D6EB5901 D6E37C02<br>C9F85701 D655A103 D7EA7402 93127D00<br>D35BA403 D4DD7902 B0A18404 D6DE7502<br>D765A903 AD928404 D0D87C02 D7E97F02<br>CD509E03 CAF85201 CFF75601 CEF45E01<br>D0E86902 D1D17F02 AD928104 AFA18304<br>D4DB5C02 88077D00 C6F75701 7D127D00<br>A99A8E04 C8609E03 D15DA503 AB957E04<br>AE9A8004 79218100 D065A103 A99E9A04 |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Программное и аппаратное обеспечение

### OS

|             |                       |
|-------------|-----------------------|
| Семейство   | Windows               |
| Версия      | 10.0.19045            |
| Релиз       | 10                    |
| Платформа   | Windows-10-10.0.19045 |
| Архитектура | AMD64                 |

### CPU

|                      |                                        |
|----------------------|----------------------------------------|
| Модель               | AMD Ryzen 5 5600H with Radeon Graphics |
| Архитектура          | AMD64                                  |
| Разрядность          | 64                                     |
| Частота (паспортная) | 3.2940 GHz                             |
| Частота (факт.)      | 3.3010 GHz                             |
| Ядер (физ.)          | 6                                      |
| Потоков (лог.)       | 12                                     |

### RAM

|          |          |
|----------|----------|
| Всего    | 15.35 GB |
| Доступно | 4.35 GB  |
| Swap     | 6.00 GB  |

### Диски

| Диск | ФС    | Всего, GB | Использовано, GB | Свободно, GB | Использовано, % |
|------|-------|-----------|------------------|--------------|-----------------|
| C:\  | NTFS  | 470.94    | 391.13           | 79.82        | 83.05           |
| D:\  | FAT32 | 0.50      | 0.00             | 0.50         | 0.00            |

### GPU

|                     |                                       |
|---------------------|---------------------------------------|
| Compute capability  | 8.6                                   |
| Name                | NVIDIA GeForce RTX 3050 Ti Laptop GPU |
| Total Global Memory | 4294443008                            |

|                         |                            |
|-------------------------|----------------------------|
| Shared memory per block | 49152                      |
| Registers per block     | 65536                      |
| Warp size               | 32                         |
| Max threads per block   | (1024, 1024, 64)           |
| Max block               | (2147483647, 65535, 65535) |
| Total constant memory   | 65536                      |
| Multiprocessor's count  | 20                         |

**Используемая IDE:** Visual Studio Code

**Версия компилятора:**

```
C:\Program Files\Microsoft Visual Studio\2022\Community>nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed Aug 20 13:58:20 Pacific Daylight Time 2025
Cuda compilation tools, release 13.0, V13.0.88
Build cuda_13.0.r13.0/compiler.36424714_0
```

## Метод решения

В программе данные считываются из бинарного файла формата int32 w, int32 h, затем RGBA-пиксели в вектор `std::vector<Pixel>`. На CPU по входным выборкам для каждого класса  $j$  накапливаются суммы  $R/G/B$  (в double) и делением на число образцов  $np_j$  получаются средние  $avg_j = (R, G, B)$ . Далее каждый  $avg_j$  нормируется по длине (деление на  $|avg_j|$ ), и плоский массив из  $nc$  нормированных троек  $(r, g, b)$  копируется в `__constant__` память видеокарты (`c_classVec`) вызовом `cudaMemcpyToSymbol`. На GPU выделяются линейные буферы `d_in` и `d_out`, исходный кадр целиком копируется в `d_in`. Запуск вычислений выполняется одномерной сеткой `kernel<<<1024,1024>>>`. Внутри ядра используется индекс  $s$  шагом `offset=blockDim.x*gridDim.x`: каждый поток обрабатывает пиксели  $i=idx$ ;  $i < totalPixels$ ;  $i += offset$  — такой способ масштабируется на любые размеры  $w \times h$  и обеспечивает коалесцированный доступ к `d_in/d_out`. Для каждого пикселя  $p = (R, G, B, A)$  вычисляются скалярные произведения  $score_c = \text{dot}( (R, G, B), c\_classVec[c] )$  по всем классам  $c$ , где `c_classVec` хранит нормированные  $avg_j$ ; выбирается индекс класса с максимальным  $score$ , и он записывается в альфа-канал выходного пикселя ( $q.a = \text{bestClass}$ ). Таким образом реализуется «метод спектрального угла»

После завершения `d_out` копируется на хост и сохраняется в файл того же бинарного формата. Все выделенные ресурсы освобождаются (`cudaFree` для `d_in/d_out`), а для всех CUDA-вызовов используется макрос `CSC` с проверкой ошибок; дополнительно контролируются ограничения `MaxPixels` и `MaxClasses` при чтении входных данных.

## Описание программы

Программа состоит из функции `main`, одного CUDA-ядра и двух вспомогательных функций для чтения/записи бинарного изображения. Для удобства работы с пикселями используется 4-байтовая структура `Pixel`, где каждый канал — однобайтовое беззнаковое целое.

### Ядро.

```
// kernel function
__global__ void kernel(const Pixel* d_in, Pixel* d_out,
                      int32_t totalPixels, int32_t nc)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int offset = blockDim.x * gridDim.x;

    for (int i = idx; i < totalPixels; i += offset) {
        Pixel p = d_in[i];

        float pr = static_cast<float>(p.r);
        float pg = static_cast<float>(p.g);
        float pb = static_cast<float>(p.b);

        int bestClass = 0;
        float bestScore =
            pr * c_classVec[0 * 3 + 0] +
            pg * c_classVec[0 * 3 + 1] +
            pb * c_classVec[0 * 3 + 2];

        for (int c = 1; c < nc; ++c) {
            float score =
                pr * c_classVec[c * 3 + 0] +
                pg * c_classVec[c * 3 + 1] +
                pb * c_classVec[c * 3 + 2];

            if (score > bestScore) {
                bestScore = score;
                bestClass = c;
            }
        }

        Pixel q = p;
        q.a = static_cast<uint8_t>(bestClass);
        d_out[i] = q;
    }
}
```

Ядро принимает указатели `const Pixel* d_in`, `Pixel* d_out`, общее число пикселей `totalPixels` и число классов `nc`. В `__constant__`-памяти заранее лежит плоский массив нормированных опорных векторов цвета `c_classVec[MaxClasses*3]` вида `[r0,g0,b0, r1,g1,b1, ...]`. Запуск выполняется одномерной конфигурацией `kernel<<<1024,1024>>>`, а внутри используется шагающий паттерн: каждый поток стартует с индекса `idx = blockIdx.x * blockDim.x + threadIdx.x` и обходит линейный буфер с шагом `offset = blockDim.x * gridDim.x` — `for (int i = idx; i < totalPixels; i += offset)`. Для каждого пикселя `p=(R,G,B,A)` вычисляются скалярные произведения со всеми опорными векторами: `score_c = R*c_r + G*c_g + B*c_b`. Номер лучшего класса записывается в альфа-канал выходного пикселя (`q.a = bestClass`), а RGB сохраняются исходными.

#### Константные данные.

```
// в константу кладем уже нормализованные средние
// храним подряд: [c0.r, c0.g, c0.b, c1.r, c1.g, c1.b, и тд]
__constant__ float c_classVec[MaxClasses * 3];
```

Массив объявлен с модификатором `__constant__`, чтобы храниться в константной памяти устройства и быстро кешироваться всеми потоками.

#### Проверка ошибок.

```
// Check CUDA call macros
#define CSC(call)
do {
    cudaError_t res = call;
    if (res != cudaSuccess) {
        fprintf(stderr, "ERROR in %s:%d. Message: %s\n",
                __FILE__, __LINE__, cudaGetErrorString(res));
        exit(0);
    }
} while(0)
```

Для всех CUDA-вызовов используется макрос `CSC(...)`, который проверяет код возврата, а при ошибке выводит файл/строку и текст сообщения драйвера.

#### Функция main.

В `main` из бинарного файла считываются `int32 w`, `int32 h` и весь кадр RGBA в `std::vector<Pixel>`; проверяется лимит `w*h <= MaxPixels`. Далее из `stdin` считываются параметры классификации: число классов `nc` и для каждого класса — количество образцов и их координаты. На CPU для каждого класса аккумулируются суммы R/G/B, усредняются, затем каждая средняя тройка нормируется по длине; в случае почти нулевой длины подставляется `[1,0,0]`. Плоский массив `h_normed[nc*3]` копируется в `__constant__` память устройства вызовом `cudaMemcpyToSymbol(c_classVec, ...)`. На GPU выделяются линейные буферы `d_in` и `d_out`, входные пиксели целиком копируются в `d_in`. Ядро запускается конфигурацией `<<<1024,1024>>>`; после `cudaDeviceSynchronize()` выход копируется обратно в вектор хоста и записывается в файл того же формата (с теми же `w,h`). Все ресурсы освобождаются (`cudaFree(d_in)`, `cudaFree(d_out)`), используется макрос `CSC` для проверки ошибок.

```

int main() {
    std::string inFilePath, outFilePath;
    std::cin >> inFilePath >> outFilePath;

    int32_t w, h;
    std::vector<Pixel> pixels;
    if (!readImage(inFilePath, w, h, pixels)) {
        return 0;
    }

    // dumpImageHex(w, h, pixels, std::cout);

    int nc = 0;
    if (!(std::cin >> nc)) {
        std::cerr << "ERROR: can't read number of classes\n";
        return 0;
    }

    if (nc <= 0 || nc > MaxClasses) {
        std::cerr << "ERROR: bad number of classes\n";
        return 0;
    }

    float h_means[MaxClasses][3] = {0.0f};

    for (int j = 0; j < nc; ++j) {
        int npj = 0;
        std::cin >> npj;

        if (npj <= 0) {
            std::cerr << "ERROR: zero samples for class " << j << "\n";
            return 0;
        }

        double sumR = 0.0;
        double sumG = 0.0;
        double sumB = 0.0;

        for (int i = 0; i < npj; ++i) {
            int x, y;
            std::cin >> x >> y;

            if (x < 0 || x >= w || y < 0 || y >= h) {
                std::cerr << "ERROR: bad sample coords for class " << j << "\n";
                return 0;
            }

            const Pixel& p = pixels[y * w + x];
            sumR += static_cast<double>(p.r);
            sumG += static_cast<double>(p.g);
            sumB += static_cast<double>(p.b);
        }

        double invN = 1.0 / static_cast<double>(npj);
        h_means[j][0] = static_cast<float>(sumR * invN);
        h_means[j][1] = static_cast<float>(sumG * invN);
        h_means[j][2] = static_cast<float>(sumB * invN);
    }
}

```

```

float h_normed[MaxClasses * 3];
for (int j = 0; j < nc; ++j) {
    float r = h_means[j][0];
    float g = h_means[j][1];
    float b = h_means[j][2];
    float len = std::sqrt(r * r + g * g + b * b);

    if (len < 1e-6f) {
        h_normed[j * 3 + 0] = 1.0f;
        h_normed[j * 3 + 1] = 0.0f;
        h_normed[j * 3 + 2] = 0.0f;
    } else {
        h_normed[j * 3 + 0] = r / len;
        h_normed[j * 3 + 1] = g / len;
        h_normed[j * 3 + 2] = b / len;
    }
}

CSC(cudaMemcpyToSymbol(c_classVec, h_normed, sizeof(float) * MaxClasses * 3, 0, cudaMemcpyHostToDevice));

Pixel* d_in = nullptr;
Pixel* d_out = nullptr;
size_t bytes = w * h * sizeof(Pixel);

CSC(cudaMalloc(&d_in, bytes));
CSC(cudaMalloc(&d_out, bytes));

CSC(cudaMemcpy(d_in, pixels.data(), bytes, cudaMemcpyHostToDevice));

int32_t totalPixels = w * h;
kernel<<<1024, 1024>>>(d_in, d_out, totalPixels, nc);
CSC(cudaGetLastError());
CSC(cudaDeviceSynchronize());

pixels.assign(totalPixels, {});
CSC(cudaMemcpy(pixels.data(), d_out, bytes, cudaMemcpyDeviceToHost));

if (!writeImage(outFilePath, w, h, pixels)) {
    cudaFree(d_in);
    cudaFree(d_out);

    return 0;
}

cudaFree(d_in);
cudaFree(d_out);

// dumpImageHex(w, h, pixels, std::cout);

return 0;
}

```



## Функции ввода/вывода.

```
// Helper functions (host)
bool readImage(const std::string& path, int32_t& w, int32_t& h, std::vector<Pixel>& pixels) {
    std::ifstream is(path, std::ios::binary);
    if (!is) {
        std::cerr << "ERROR: can't read input file!\n"; return false;
    }

    is.read(reinterpret_cast<char*>(&w), sizeof(int32_t));
    is.read(reinterpret_cast<char*>(&h), sizeof(int32_t));

    if (w <= 0 || h <= 0 || w >= MaxValueWH || h >= MaxValueWH) {
        std::cerr << "ERROR: width or height wrong value!\n"; return false;
    }
    // static_cast to avoid overflow
    if (static_cast<int64_t>(w) * static_cast<int64_t>(h) > MaxPixels) {
        std::cerr << "ERROR: more pixels in the image than allowed!\n"; return false;
    }

    pixels.resize(w * h);
    is.read(reinterpret_cast<char*>(pixels.data()), pixels.size() * sizeof(Pixel));

    return true;
}

bool writeImage(const std::string& path, int32_t w, int32_t h, const std::vector<Pixel>& pixels) {
    std::ofstream os(path, std::ios::binary | std::ios::trunc);
    if (!os) {
        std::cerr << "ERROR: can't open output file!\n"; return false;
    }

    os.write(reinterpret_cast<const char*>(&w), sizeof(int32_t));
    os.write(reinterpret_cast<const char*>(&h), sizeof(int32_t));
    os.write(reinterpret_cast<const char*>(pixels.data()), pixels.size() * sizeof(Pixel));

    return true;
}
```

Чтение и запись реализованы прямой работой с файлами C++ в бинарном режиме: сначала два `int32` (ширина и высота), далее `w*h` структур `Pixel`. В функциях присутствуют проверки габаритов и числа пикселей на соответствие ограничениям работы.

Для локально проверки работы программы были также реализованы функции дампа бинарных файлов в формате совпадающим с условием лабораторной работы.

```
// HEX file dump
inline void printByteHex(uint8_t b, std::ostream& os) {
    os << std::hex << std::setw(2) << std::setfill('0') << (unsigned)b;
    os << std::dec;
}

inline void printU32LE(uint32_t v, std::ostream& os) {
    printByteHex((uint8_t)(v & 0xFF), os);
    printByteHex((uint8_t)((v >> 8) & 0xFF), os);
    printByteHex((uint8_t)((v >> 16) & 0xFF), os);
    printByteHex((uint8_t)((v >> 24) & 0xFF), os);
}

void dumpImageHex(int32_t w, int32_t h, const std::vector<Pixel>& pixels, std::ostream& os) {
    printU32LE((uint32_t)w, os); os << ' ';
    printU32LE((uint32_t)h, os); os << '\n';

    for (int32_t y = 0; y < h; ++y) {
        for (int32_t x = 0; x < w; ++x) {
            const Pixel& p = pixels[(size_t)y * (size_t)w + (size_t)x];
            printByteHex(p.r, os);
            printByteHex(p.g, os);
            printByteHex(p.b, os);
            printByteHex(p.a, os);
            if (x + 1 < w) os << ' ';
        }
        os << '\n';
    }
}
```

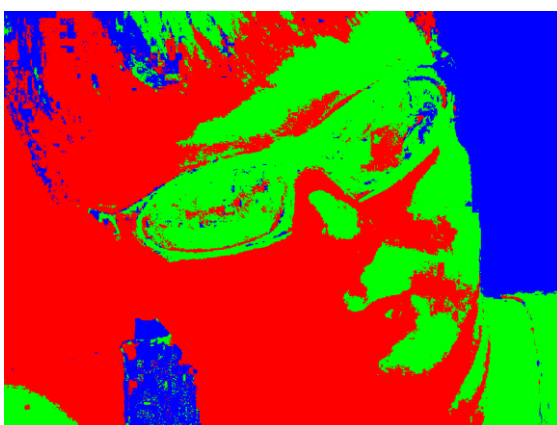
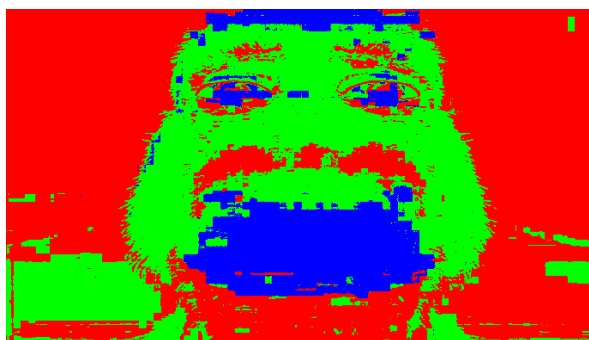
## Результаты

Для измерения производительности было использовано изображение 1280x960 пикселей

| Конфигурация     | Время, ms       |
|------------------|-----------------|
| <<<1024, 1024>>> | <b>0.236915</b> |
| <<<512, 512>>>   | <b>0.270543</b> |
| <<<256, 1>>>     | <b>4.943612</b> |
| <<<64, 4>>>      | <b>2.010938</b> |
| <<<8, 2>>>       | <b>8.430824</b> |

CPU time = **49 ms**

## Примеры обработанных изображений



## Выводы

В результате выполненной работы были улучшены навыки обработки изображений с использованием GPU. Реализован алгоритм Спектрального угла для классификации и кластеризации изображений, что позволило на практике закрепить принципы программирования на языке C++.

Реализация, выполненная в рамках данной работы, носит учебный характер и предназначена для демонстрации принципов параллельной обработки изображений на GPU. Несмотря на то, что код может быть оптимизирован для повышения производительности и расширения функциональности, разработанная программа позволяет наглядно изучить особенности вычислений на графических процессорах.

Метод спектрального угла применяется в ДЗЗ, компьютерном зрении, мониторинге измерений и ряде других задач.

Экспериментальные замеры на изображении 1280×960 показали существенное преимущество GPU над CPU: при лучшей конфигурации время GPU составило 0.2369 мс против 49 мс на CPU, что даёт ускорение порядка. Почти равноценный результат показала конфигурация <<<512, 512>>> — 0.2705 мс. Более мелкие/несбалансированные сетки заметно проигрывают по времени из-за худшей утилизации варпов и меньшей способности скрывать латентность памяти: <<<64, 4>>> — 2.0109 мс, <<<256, 1>>> — 4.9436 мс, <<<8, 2>>> — 8.4308 мс.

Максимальная производительность достигается при конфигурациях с большим числом активных потоков и размерах блоков, кратных 32 (размеру варпа), что обеспечивает высокую заполняемость мультипроцессоров и эффективноекрытие задержек памяти. CPU-реализация значительно уступает и может рассматриваться лишь как эталон корректности и базовая точка сравнения по скорости.